**Introduction to Web Development**

Learning Hub Course: Full Stack Web Development for Beginners Module: Introduction to Full Stack Web Development Lesson: 01 – Introduction to Web Development Level: Beginner Estimated Duration: 45–60 minutes

**1. Learning Objectives**

By the end of this lesson, you will be able to:

- Define web development.
- Understand the difference between frontend and backend development.
- Explain the role of databases in web applications.
- Understand how a browser communicates with a web server.
- Identify the basic technologies used in web development.
- Create a simple HTML webpage.
- Understand responsive design and basic web security.

**2. What Is Web Development?**

Web development is the process of creating and maintaining websites and web applications that run in a web browser.

Examples include:

- Educational websites
- Online shopping websites
- Banking applications
- Social media platforms
- Learning Management Systems (LMS)
- Online booking systems

For example, Learning Hub is a web application where students can register, log in, access courses, view lessons, and complete quizzes.

Web development generally includes three main areas:

1. Frontend development

2. Backend development
3. Database management

**3. Frontend Development**

Frontend development focuses on the parts of a website that users can see and interact with.

Examples include:

- Navigation menus
- Buttons
- Forms
- Images and text
- Course cards
- Login pages
- Dashboards

Frontend development commonly uses three technologies.

**3.1 HTML**

HTML stands for HyperText Markup Language.

HTML defines the structure and content of a webpage.

Examples of HTML elements include:

- Headings
- Paragraphs
- Links
- Images
- Forms
- Buttons

**3.2 CSS**

CSS stands for Cascading Style Sheets.

CSS controls the appearance and layout of a webpage.

It can be used to change:

- Colors
- Fonts
- Backgrounds
- Spacing
- Borders
- Page layouts
- Responsive behavior

**3.3 JavaScript**

JavaScript adds interactive behavior to webpages.

Examples include:

- Validating forms
- Opening menus
- Displaying alerts
- Updating page content
- Handling button clicks
- Communicating with APIs

**4. Backend Development**

Backend development handles the operations that take place behind the scenes.

The backend commonly manages:

- User registration and authentication
- Business logic
- Data processing
- Permissions and authorization
- Communication with databases

- API requests
- File uploads

Popular backend technologies include:

- PHP
- Java with Spring Boot
- JavaScript with Node.js
- Python with Django or FastAPI
- C# with ASP.NET Core

For example, when a student logs in to Learning Hub, the backend receives the login request, validates the credentials, and returns an appropriate response.

Important: Password verification and other security-sensitive checks must be performed on the server, not only in JavaScript.

**5. What Is a Database?**

A database is an organized collection of data that can be stored, retrieved, updated, and deleted.

A Learning Management System might store:

- User accounts
- Instructor details
- Course information
- Modules and lessons
- Quiz questions
- Student progress
- Course reviews

Popular database systems include:

- MySQL
- MariaDB
- PostgreSQL

- MongoDB
- Microsoft SQL Server

For example, MySQL or MariaDB can store the courses and student accounts used by a PHP-based Learning Hub application.

**6. How Frontend, Backend, and Database Work Together**

Consider a student logging in to an LMS.

1. The student enters an email address and password on the login page.
2. The frontend sends the login information to the backend over HTTPS.
3. The backend validates the submitted information.
4. The backend retrieves the relevant user record from the database.
5. The password is verified against its stored password hash.
6. The backend determines whether the login is successful.
7. The frontend displays a response or redirects the student to the dashboard.

Each part has a different responsibility.

- Frontend: Displays the login form and results.
- Backend: Processes the login request and enforces security.
- Database: Stores user records and related information.

**7. How a Website Works**

When a user visits a website, several steps take place.

1. The user enters a website address into the browser.
2. The browser uses DNS to find the server's network address.
3. The browser establishes a connection to the server, typically using HTTPS.
4. The browser sends an HTTP request.
5. The server processes the request.
6. The server returns an HTTP response.
7. The browser renders the HTML, applies CSS, and executes JavaScript.

The browser and server communicate using HTTP or HTTPS.

HTTP stands for Hypertext Transfer Protocol.

HTTPS is HTTP protected by encryption using TLS. It helps protect data while it travels between the browser and server.

**8. Static and Dynamic Websites**

**8.1 Static Websites**

A static website generally serves predefined files to visitors.

Characteristics:

- Often uses HTML and CSS.
- May include JavaScript.
- Can be hosted with relatively simple infrastructure.
- Does not necessarily require a database.

Examples include simple portfolios, informational pages, and documentation sites.

**8.2 Dynamic Websites**

A dynamic website can generate or update content based on user actions, stored data, or application logic.

Characteristics:

- May use a backend server.
- Often connects to a database.
- Can support user accounts.
- Can display personalized information.
- Can process forms and transactions.

Examples include online stores, banking applications, and Learning Management Systems.

A website can contain both static and dynamic features.

**9. Your First HTML Webpage**

The following example creates a simple webpage using HTML, CSS, and JavaScript.

Save the code as index.html.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Webpage</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      text-align: center;
      padding: 40px;
    }

    h1 {
      color: #2563eb;
    }

    button {
      padding: 10px 20px;
      border: none;
      border-radius: 5px;
      background-color: #2563eb;
      color: white;
      cursor: pointer;
```

```
    }
  </style>
</head>

<body>
  <h1>Welcome to Web Development</h1>

  <p>This is my first webpage.</p>

  <button onclick="showMessage()">
    Click Me
  </button>

  <script>
    function showMessage() {
      alert("Welcome to web development!");
    }
  </script>
</body>
</html>
```

**Explanation**

- <!DOCTYPE html> declares an HTML document.
- <html> contains the webpage.
- <head> contains metadata, the page title, and styles.
- <body> contains the visible content.
- <h1> creates a main heading.

- <p> creates a paragraph.
- <button> creates a clickable button.
- <style> contains CSS rules.
- <script> contains JavaScript code.
- showMessage() displays an alert when the button is clicked.

**How to Run the Example**

1. Open Visual Studio Code or another text editor.
2. Create a file named index.html.
3. Copy the code into the file.
4. Save the file.
5. Open the file in Google Chrome or another web browser.
6. Click the button to test the JavaScript functionality.

**10. Essential Web Development Tools**

**10.1 Code Editor**

A code editor helps developers write and manage source code.

Examples include:

- Visual Studio Code
- Notepad++
- Sublime Text

Visual Studio Code is a useful choice for beginners.

**10.2 Web Browser**

A browser allows developers to view and test webpages.

Examples include:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Browser Developer Tools help developers inspect HTML, CSS, JavaScript errors, network requests, and webpage performance.

**10.3 Git and GitHub**

Git is a version control system that tracks changes to source code.

GitHub is an online platform for hosting Git repositories and collaborating with other developers.

Developers use Git and GitHub to:

- Track code changes
- Work with other developers
- Manage project versions
- Share projects
- Maintain a portfolio

**10.4 XAMPP**

XAMPP provides a local development environment that includes Apache, MariaDB, PHP, and other components.

It is useful when learning PHP and database-driven web applications.

For example, a PHP project can be stored inside the XAMPP htdocs directory and accessed through a local URL.

**11. Responsive Web Design**

Responsive web design allows a website to adapt to different screen sizes, including desktop computers, tablets, and mobile phones.

A responsive website should be readable and usable on different devices.

The following CSS example changes the page padding on smaller screens.

```
body {
  padding: 40px;
}
```

```
@media (max-width: 600px) {
  body {
    padding: 15px;
  }
}
```

The @media rule applies specific CSS styles when the screen width meets the given condition.

The viewport metadata in the HTML example also helps the browser display the webpage at an appropriate scale on mobile devices.

**12. Web Accessibility**

Web accessibility means designing websites that people with different abilities can use.

Good accessibility practices include:

- Using meaningful headings.
- Providing labels for form fields.
- Using descriptive text for meaningful images.
- Maintaining sufficient color contrast.
- Supporting keyboard navigation.
- Providing clear error messages.
- Using semantic HTML elements.

Accessible websites are generally easier for everyone to understand and use.

**13. Basic Web Security**

Security is an important part of web development.

Developers should follow these practices.

**13.1 Use HTTPS**

HTTPS encrypts communication between the browser and the server when correctly configured.

**13.2 Validate User Input**

Validate submitted data on the server. Client-side validation can improve usability, but it must not be the only validation.

### 13.3 Use Prepared Statements

Prepared statements help protect database queries against SQL injection when used correctly.

### 13.4 Hash Passwords

Never store plain-text passwords. Use established password-hashing functions such as PHP's password_hash() and verify passwords with password_verify().

### 13.5 Check Authorization

A user should only be allowed to access resources and actions permitted by their role and ownership.

For example, an instructor should not be able to edit another instructor's course simply by changing a course ID in a URL.

### 13.6 Protect Sensitive Information

Do not expose database passwords, API secrets, private keys, or other sensitive credentials in frontend JavaScript or public repositories.

## 14. Common Beginner Mistakes

Some common mistakes in web development include:

1. Writing HTML without proper structure.
2. Forgetting to close HTML elements where required.
3. Using incorrect file paths.
4. Ignoring responsive design.
5. Testing only on one browser or device.
6. Relying entirely on client-side validation.
7. Storing passwords as plain text.
8. Writing SQL queries without appropriate protection.
9. Ignoring browser console errors.
10. Failing to test forms and links before publishing a website.

Developers improve by practicing, testing their code, reading error messages, and fixing problems step by step.

**15. Practical Activities**

Complete the following activities to improve your understanding.

**Activity 1: Create a Personal Webpage**

Create an HTML page containing:

- Your name
- A short introduction
- Your education
- Your technical skills
- Your career goals

**Activity 2: Add CSS Styling**

Change the webpage's:

- Background color
- Heading color
- Font size
- Spacing
- Button design

**Activity 3: Add JavaScript**

Create a button that displays a welcome message when clicked.

**Activity 4: Make the Page Responsive**

Use a CSS media query to adjust the layout for smaller screens.

**Activity 5: Explore Developer Tools**

Open the webpage in Chrome and use Developer Tools to inspect its HTML, CSS, and JavaScript console.

**16. Quick Review Questions**

1. What is web development?

2. What is the purpose of HTML?
3. What is the difference between HTML and CSS?
4. What is JavaScript used for?
5. What is frontend development?
6. What is backend development?
7. Why do web applications use databases?
8. What happens when a browser sends an HTTP request?
9. What is the difference between a static website and a dynamic website?
10. Why is responsive web design important?
11. Why should passwords be hashed?
12. Why is server-side validation necessary?

**17. Summary**

Web development is the process of building and maintaining websites and web applications.

HTML defines webpage structure, CSS controls presentation, and JavaScript adds interactivity. Backend technologies process requests and implement application logic, while databases store and manage information.

A browser communicates with a server through HTTP or HTTPS. Developers use code editors, browsers, Git, GitHub, and local development tools to build and test websites.

Responsive design, accessibility, input validation, password hashing, and proper authorization are essential considerations when developing reliable web applications.

The best way to learn web development is to practice by building small projects and gradually increasing their complexity.